

Augmenting the Generality and Performance of Large Language Models for Software Engineering

Fabian C. Peña

Faculty of Computer Science and Mathematics

University of Passau

fabiancamilo.penalozano@uni-passau.de

Abstract—Large Language Models (LLMs) are revolutionizing software engineering (SE), with special emphasis on code generation and analysis. However, their applications to broader SE practices including conceptualization, design, and other non-code tasks, remain partially underexplored. This research aims to augment the generality and performance of LLMs for SE by (1) advancing the understanding of how LLMs with different characteristics perform on various non-code tasks, (2) evaluating them as sources of foundational knowledge in SE, and (3) effectively detecting hallucinations on SE statements. The expected contributions include a variety of LLMs trained and evaluated on domain-specific datasets, new benchmarks on foundational knowledge in SE, and methods for detecting hallucinations. Initial results in terms of performance improvements on various non-code tasks are promising.

Index Terms—Large Language Model (LLM), software engineering, benchmarking, hallucination detection.

I. INTRODUCTION

Research and applications in Natural Language Processing (NLP) have increased significantly with the arrival of the transformer architecture [1], followed by some notable neural network models such as BERT [2], GPT [3], and T5 [4]. Nowadays, these and subsequent models are more commonly known as Large Language Models (LLMs). LLMs are designed to predict tokens (words, symbols, etc.) through a statistical process learned from data, emerging in higher-level capabilities or downstream tasks. In particular in the software engineering (SE) industry, LLMs have become popular, enabling tools such as GitHub Copilot [5] and Devin [6] to assist developers in tasks related to code generation and analysis.

Newer LLMs tend to be larger and trained for longer on more extensive datasets, typically resulting in better performance [7], [8]. For SE, learning from large amounts of source code, in different programming languages accompanied by natural language text with educational or industrial purposes, has emerged in capabilities such as writing code from free-text instructions or automatically generating technical documentation. Currently, these capabilities are partially evaluated using public benchmarks that compile thousands of coding problems with the corresponding unit tests for direct verification [9]–[12].

However, SE encompasses more than just writing or understanding code. The typical SE life cycle involves stages such as requirement analysis, software design, software development, quality assurance, and software maintenance. Consequently,

researchers have also been concerned with building and evaluating LLMs across a broader range of SE tasks, as highlighted in recent surveys [13]–[15]. Despite the variety of datasets, network architectures, model sizes, prompting, and fine-tuning techniques documented in these surveys, there is a high imbalance toward coding-related tasks. Furthermore, there are no works focused on evaluating the foundational knowledge (i.e., factual and conceptual knowledge [16]) of LLMs in SE.

Building on the ideas presented in [17], [18] regarding the level of applicability that LLMs can potentially achieve, to effectively support SE practitioners in real-world scenarios, a certain level of generality (i.e., range of supported tasks) and performance are required. To satisfy this dual need, to the best of our knowledge, developers of LLM-based systems tend to use the most powerful general-purpose LLMs available on the market (e.g., GPT-4o, Claude 3.5, Llama 3.2), assuming them as the best choice for any type of task. We argue that this practice is far from optimal, since similar or better performance could potentially be achieved with smaller LLMs using different network architectures and trained on domain-specific datasets.

Another concern when working with LLMs is their risk of generating hallucinations (i.e., plausible but factually incorrect or nonsensical statements [19]). Reducing this risk as much as possible is also essential to build trust among SE practitioners. While methods to detect and mitigate hallucinations have been proposed [20], [21], none specifically target SE.

II. RESEARCH QUESTIONS

On the path to generality and performance of LLMs for SE, with the assumption that achieving these properties should not be the goal of a single LLM from a system perspective, we identify opportunities to (1) advance the understanding of how LLMs with different pre-training datasets, network architectures, and model sizes perform on various non-code tasks, (2) build new benchmarks to evaluate LLMs as sources of foundational knowledge in SE, and (3) develop methods to effectively detect hallucinations on SE statements generated by LLMs. To address these opportunities, we propose the following research questions:

Main research question:

- 1) How much can the generality and performance of LLMs be augmented to cover a broader range of non-code and foundational knowledge SE-related tasks?

Secondary research questions:

- 2) What is the impact of different domain-specific datasets, network architectures, and model sizes on the performance of LLMs for various non-code tasks identified in the literature?
- 3) Under what scenarios can LLMs accurately provide foundational knowledge in SE in terms of terminology and common object-oriented design scenarios?
- 4) Can effective methods be developed to detect hallucinations on SE statements generated by LLMs?

In recent years, LLM-based systems (a.k.a LLM agents) have gained important attention from the community as an alternative to everything that an LLM cannot achieve individually [22]. LLM agents tend to outperform single LLMs on different tasks [23], [24] and, while the opportunities for these agents are extensive, fundamental capabilities of LLMs for SE at the most granular level remain partially underexplored. We argue that continuing to push for improvements in LLM agents with a strong focus only on coding-related tasks is potentially dangerous for new generations of SE practitioners and goes against what the software industry stands for.

III. EXPECTED CONTRIBUTIONS

As part of this proposal, we expect to make four key contributions. Each contribution is accompanied by a peer-reviewed paper and a replication kit containing source code, datasets, and models. Network architecture, model size, training dataset, and refinement technique (i.e., prompting and fine-tuning, where applicable) are consistently considered as dimensions of analysis.

A. LLMs trained on SE datasets

Based on common architectures like RoBERTa [25], GPT-2 [26], and T5 [4], among others, new LLMs are pre-trained from scratch and fine-tuned from publicly available checkpoints on over 200 GB of texts sourced from platforms such as GitHub, Stack Overflow, JIRA, and ArXiv. Subsequently, these LLMs are fine-tuned using a supervised approach on various non-code tasks identified in the literature. The performance on these tasks is evaluated using established metrics tailored to the specific task type. For benchmarking purposes, baseline models using machine learning algorithms like XGBoost and FastText are also trained. With these results we expect to contribute to the first and second research questions.

B. Benchmarks on foundational knowledge in SE

To establish two new benchmarks on foundational knowledge in SE, terminology is extracted from several Standards Development Organizations (SDOs) [27]–[30], and a set of common object-oriented design scenarios with their corresponding UML diagrams are defined. LLMs trained in Section III-A are evaluated using these benchmarks in different settings. First, the ability to accurately discriminate between (i) terms and definitions, and (ii) design scenarios and UML diagrams. The main approach considered for the discrimination is zero-shot classification, which does not require specific

fine-tuning of LLMs, and the evaluation is based on common classification metrics. Second, the performance in generating SE definitions and UML diagrams in formats like PlantUML¹. Because LLMs may not be trained on the PlantUML syntax, fine-tuning or prompting techniques are optionally considered. The evaluation is based on metrics like Bilingual Evaluation Understudy (BLEU), additionally requiring a qualitative analysis to identify common reasons for errors. With these results we expect to contribute to the first and third research questions.

C. Generation of nonsensical SE statements

Building upon the benchmarks established in Section III-B, a complementary benchmark of nonsensical SE statements is generated. Each entry consists of the original statement, a systematically modified nonsensical version, and the type of modification applied (e.g., replacement with antonyms, addition of an unsuitable negation, etc.). LLMs from Section III-A are prompted with these nonsensical statements to generate responses that may further propagate or elaborate on the nonsensical content. The evaluation primarily involves a qualitative assessment to determine if generation of nonsensical statements stems from deductive reasoning and if the underlying reasons align with the modifications applied to the prompts. The original statements are used as a control group. With these results we expect to contribute to the first and fourth research questions.

D. Detection of nonsensical SE statements

Utilizing the benchmark generated in Section III-C, an experimental process based on a zero-shot classification approach to detect nonsensical statements is conducted. This process involves classifying the original statements, the manually modified nonsensical statements, and the generated statements. The evaluation is carried out in multiple stages. First, for the original and modified statements, standard evaluation metrics are calculated. Second, for the generated statements, the label probabilities assigned by the LLM are compared to those assigned to the original and modified statements. The comparison is done by using techniques like QQ-plots and Kolmogorov-Smirnov test. Additionally, the analysis is complemented using a segmented approach based on the type of modification applied. With these results we expect to contribute to the first and fourth research questions.

IV. INITIAL RESULTS

To date, BERT, RoBERTa, and GPT-2 models have been pre-trained from scratch and fine-tuned from publicly available checkpoints on more than 23 GB of textual data. Additionally, these models have also been fine-tuned on labeled datasets for 17 non-code tasks, showing an increase in performance in most cases. The first paper is being prepared for publication.

ACKNOWLEDGMENTS

This work is funded by the German Research Foundation (DFG) through the project SENLP, grant 524228075.

¹<https://plantuml.com/>

REFERENCES

- [1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones *et al.*, "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30. Curran Associates, Inc., 2017.
- [2] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*. Minneapolis, Minnesota: Association for Computational Linguistics, Jun. 2019, pp. 4171–4186.
- [3] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan *et al.*, "Language models are few-shot learners," in *Advances in Neural Information Processing Systems*, vol. 33. Curran Associates, Inc., 2020, pp. 1877–1901.
- [4] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang *et al.*, "Exploring the limits of transfer learning with a unified text-to-text transformer," *Journal of Machine Learning Research*, vol. 21, no. 140, pp. 1–67, 2020.
- [5] GitHub, "The world's most widely adopted ai developer tool," <https://github.com/features/copilot>, accessed: 2024-09-11.
- [6] C. AI, "An applied ai lab building end-to-end software agents," <https://www.cognition.ai/>, accessed: 2024-09-11.
- [7] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess *et al.*, "Scaling laws for neural language models," *arXiv preprint arXiv:2001.08361*, 2020.
- [8] J. Hoffmann, S. Borgeaud, A. Mensch, E. Buchatskaya, T. Cai *et al.*, "Training compute-optimal large language models," in *Proceedings of the 36th International Conference on Neural Information Processing Systems*, ser. NIPS '22. Red Hook, NY, USA: Curran Associates Inc., 2024.
- [9] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto *et al.*, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.0337*, 2021.
- [10] J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski *et al.*, "Program synthesis with large language models," *arXiv preprint arXiv:2108.07732*, 2021.
- [11] J. Liu, C. S. Xia, Y. Wang, and L. Zhang, "Is your code generated by chatGPT really correct? rigorous evaluation of large language models for code generation," in *Thirty-seventh Conference on Neural Information Processing Systems*, 2023.
- [12] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei *et al.*, "SWE-bench: Can language models resolve real-world github issues?" in *The Twelfth International Conference on Learning Representations*, 2024.
- [13] X. Hou, Y. Zhao, Y. Liu, Z. Yang, K. Wang *et al.*, "Large language models for software engineering: A systematic literature review," *arXiv preprint arXiv:2308.10620*, 2023.
- [14] Q. Zhang, C. Fang, Y. Xie, Y. Zhang, Y. Yang *et al.*, "A survey on large language models for software engineering," *arXiv preprint arXiv:2312.15223*, 2023.
- [15] A. Fan, B. Gokkaya, M. Harman, M. Lyubarskiy, S. Sengupta *et al.*, "Large language models for software engineering: Survey and open problems," in *2023 IEEE/ACM International Conference on Software Engineering: Future of Software Engineering (ICSE-FoSE)*. Los Alamitos, CA, USA: IEEE Computer Society, may 2023, pp. 31–53.
- [16] D. R. Krathwohl, "A revision of bloom's taxonomy: An overview," *Theory Into Practice*, vol. 41, no. 4, pp. 212–218, 2002.
- [17] M. R. Morris, J. Sohl-Dickstein, N. Fiedel, T. Warkentin, A. Dafoe *et al.*, "Position: Levels of AGI for operationalizing progress on the path to AGI," in *Proceedings of the 41st International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 235. PMLR, 21–27 Jul 2024, pp. 36 308–36 321.
- [18] M. Suleyman and M. Bhaskar, *The Coming Wave: Technology, Power, and the 21st Century's Greatest Dilemma*. Crown, 2023.
- [19] Z. Xu, S. Jain, and M. Kankanhalli, "Hallucination is inevitable: An innate limitation of large language models," *arXiv preprint arXiv:2401.11817*, 2024.
- [20] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su *et al.*, "Survey of hallucination in natural language generation," *ACM Computing Surveys*, vol. 55, no. 12, mar 2023.
- [21] L. Huang, W. Yu, W. Ma, W. Zhong, Z. Feng *et al.*, "A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions," *arXiv preprint arXiv:2311.05232*, 2023.
- [22] H. Jin, L. Huang, H. Cai, J. Yan, B. Li, and H. Chen, "From llms to llm-based agents for software engineering: A survey of current, challenges and future," *arXiv preprint arXiv:2408.02479*, 2024.
- [23] Y. Wang, W. Zhong, Y. Huang, E. Shi, M. Yang *et al.*, "Agents in software engineering: Survey, landscape, and vision," *arXiv preprint arXiv:2409.09030*, 2024.
- [24] J. Liu, K. Wang, Y. Chen, X. Peng, Z. Chen *et al.*, "Large language model-based agents for software engineering: A survey," *arXiv preprint arXiv:2409.02977*, 2024.
- [25] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi *et al.*, "Roberta: A robustly optimized BERT pretraining approach," *CoRR*, vol. abs/1907.11692, 2019.
- [26] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language models are unsupervised multitask learners," 2019.
- [27] *ISO/IEC/IEEE International Standard - Systems and software engineering—Vocabulary*, International Organization for Standardization, International Electrotechnical Commission, Institute of Electrical and Electronics Engineers Std. ISO/IEC/IEEE 24 765:2017(E), 2017.
- [28] ISTQB, "ISTQB Glossary," <https://glossary.istqb.org/>, [Accessed: Sep. 10, 2024].
- [29] IREB, "CPRE Glossary," <https://www.ireb.org/>, [Accessed: Sep. 10, 2024].
- [30] ISAQB, "CPSA Certifications - Foundation Level," <https://www.isaqb.org/>, [Accessed: Sep. 10, 2024].